Расширения 1F и 2F

Это команды MK61s, а не реконструкция микропрограмм исходного МК-61. Они исполняются в режиме памяти 112 шагов + RF и в `auto`, если программа содержит префикс как команду. В принудительном классическом режиме 105 шагов остаётся поведение ПЗУ МК-61. После обычной двухбайтовой команды перехода или вызова (51, 53, 57…5E) байт 1F/2F — её аргумент, а не префикс. Исходное ПЗУ трактует такие аргументы как шаги 25 и 35 соответственно; расширенный декодер обычного ПП сохраняет это правило.

Ассемблерные обозначения

`asm`, `isa` и `lasm` используют `far` для `1F`, `disp` для `2F`, `xmsr` для обмена Ms с регистрами R0...R8 (55) и `xmsp` для обмена Ms с программной памятью (56). Синонимы K1 и K2 также допустимы в `asm`. Для RF доступны `stoF` (4F) и `ldF` (6F).

Префикс - отдельная мнемоника, остальные байты записываются после неё:

```
asm 0000 far jmp 02 50
asm 0004 far call[A]
asm 0006 far xmsp 18
asm 0009 disp 2A disp 0D disp 53
```

Эти строки записывают соответственно `1F 51 02 50`, `1F AA`, `1F 56 18` и `2F 2A 2F 0D 2F 53`. Байты адреса `02 50` имеют смысл BCD-числа 0250; hex-номер банка `18` означает банк 24. Подкоды `disp` вводятся hex-байтами: например, `disp 53` публикует кадр, а не вызывает подпрограмму.

`lasm` распознаёт длины этих последовательностей в расширенном режиме, показывает все аргументы у префикса и не выдаёт им отдельные мнемоники. В принудительном режиме 105 шагов префиксный разбор выключен.

1F: абсолютная адресация

Программа разделена на банки по 112 байтов. Адрес `0000…9999` означает банк = адрес / 112, шаг = адрес % 112; последний банк неполный. Банк 0 — обычная память программы. Остальные банки выделяются по мере записи или перехода. 55/56 продолжают работать с текущим банком. На STM32F401 и STM32F411 одновременно могут быть заполнены 32 банка (включая банк 0). Адресовать можно любой из 90 банков диапазона, но при исчерпании свободных слотов запись или переход завершается ошибкой расширения. Очистка программы освобождает все слоты. Незаписанные байты равны `00`: так читается ещё не выделенный банк и так заполняется новый, в том числе при повторном использовании освобождённого слота. Чтение пустого банка не занимает слот. Код `00` означает ввод цифры 0; если нужна остановка, явно запишите `50` (С/П).

Последовательность	Действие
`1F 51 HH LL`	Безусловный переход
`1F 53 HH LL`	Вызов подпрограммы
`1F 57…5E HH LL`	Дальняя условная команда с условием указанного кода
`1F 7n/8n/9n/An/Cn/En`	Дальняя косвенная команда с регистром Rn
`1F 56 nn`	Обмен Ms с первыми 63 байтами банка nn
`1F 4n HH LL`	Записать целую константу BCD 0000…9999 в Rn, сохранив X и стек
`1F Bn`	Прочитать байт по адресу RF + Rn в X
`1F Dn`	Прочитать 24-битное слово little-endian по адресу RF + 3*Rn в X

`HH LL` — четыре десятичные цифры адреса в двух BCD-байтах: `1F 51 02 50` переходит на адрес 0250. Косвенная команда берёт абсолютный адрес из регистра. Для `R0…R3` и `R4…R6` сохраняется автоматическое уменьшение/увеличение регистра. При ложном условии выполняется команда сразу после всей

последовательности 1F.

Условия по X проверяют окончательный результат предыдущей арифметической команды. Вставлять NOP между арифметикой и прямым или косвенным условием 1F не требуется. Например, после 0−19 команда 1F 59 HH LL выполняет переход по отрицательному X. Это поведение исправлено в сборке 544269E1; более раннее ядро могло проверить знак до завершения записи результата ПЗУ.

Вызов сохраняет адрес после четырёх- или двухбайтовой команды. Обычные ПП, косвенные и дальние вызовы используют общий расширенный стек глубиной 64; В/О возвращает и через границу банка. Некорректный адрес, неподдерживаемый подкод или переполнение стека останавливает программу и устанавливает внутренний флаг ошибки расширения. Для дальних FL0…FL3 счётчик должен быть целым в диапазоне 1…9999: при нуле расширение останавливает программу вместо исторического десятичного переполнения регистра.

Дальние FL0…FL3 сохраняют числовые эффекты ПЗУ: представление счётчика, состояние числового ввода при продолжении цикла и синхронизацию X → X2 при выходе. Это исправлено в исходниках 23.09.2026. Прежний обработчик уменьшал счётчик средствами C++, но не воспроизводил скрытое состояние; последующие . и ВП могли дать другой результат, чем после обычного FL.

Возврат через расширенный стек сохраняет числовые эффекты обычного В/О: копирование X в скрытый регистр X2 и нормализацию X, включая ошибку при переполнении порядка. Это исправлено в исходниках 23.09.2026. Ранее такой возврат выполнял только переход; последующая . могла восстановить старое значение X2. Проверены прямые, косвенные, ближние, дальние и вложенные вызовы.

Обычные ПП и К ПП n используют преобразование локального адреса самим ПЗУ, сохраняя адрес возврата в расширенном стеке. Поэтому для них работают дробные и отрицательные указатели, автоматическое изменение R0…R6 и тёмные адреса, как у обычных БП и К БП n. Например, π² в RE даёт локальный адрес 09, а π^π — 36. Совместимость исправлена в исходниках 23.09.2026; прежний перехват К ПП n ошибочно требовал целый неотрицательный указатель меньше 112. Для дальних команд 1F сохраняется отдельный контракт: целый неотрицательный абсолютный адрес в пределах 0000…9999.

Обмен Ms с выбранным банком

1F 56 nn занимает три байта. nn — двоичный номер банка, а не BCD-адрес: например, 1F 56 18 обменивает Ms с банком 24, адресами 2688…2750. Ровно 63 байта меняются местами, включая служебные тетрады девяти страниц; байты банка 63…111, регистры R0…RF и арифметический стек сохраняются. Исполнение продолжается за трёхбайтовой командой без перехода в выбранный банк. Если выбран текущий банк, результат соответствует обычной 56, но следующая команда находится через три байта. При изменении текущего банка следующий код читается уже из обновлённой памяти.

Допустимы полные банки 0…88 (00…58 hex). Последний банк 89 содержит только 32 адресуемых байта и для обмена 63 байт не подходит. Пустой банк считается нулевым и выделяется из общего пула 32 слотов. Неверный номер или нехватка слотов останавливают программу с ошибкой расширения до изменения Ms и содержимого банков. Это относится и к слоту, необходимому для продолжения за границей банка; сама команда может пересекать границу.

Для работы с девятью числовыми полями банка 24:

```
1F 56 18 55       открыть: поля банка → R0…R8, рабочие регистры → Ms
…                 вычисления с открытой страницей
55 1F 56 18       закрыть: вернуть регистры, записать поля и восстановить Ms
```

Обычные 55 и 56 сохраняют прежнее действие. Новый обмен добавлен 25.09.2026; для программ с 1F 56 nn требуется прошивка с этой командой.

Константы и таблицы данных

1F 4n HH LL занимает четыре байта. Константа кодируется так же, как абсолютный адрес: 1F 4F 12 34 записывает 1234 в RF. Изменяется только указанный регистр; X, стек и режим числового ввода сохраняются.

1F Bn и 1F Dn занимают два байта. База RF и индекс Rn должны быть неотрицательными целыми меньше 10000, всё чтение должно помещаться в 0000…9999. Слово собирается из трёх байтов, младший первым, до 16777215. Чтение может пересекать банки; невыделенная память возвращает нули. Меняется только X; регистры, X1/Y/Z/T, курсор и экран сохраняются. Команды не имитируют подъём стека или ввод цифр. Для сохранения результата можно сразу выполнить обычное X→П n; перед вводом новой константы используйте обычный В↑, если это требуется состоянием числового ввода. Проверка завершается до изменения X, в том

числе при Rn=RF.

Например, RF=200 и R7=4: `1F B7` читает байт 204, а `1F D7` — байты 212…214. Неверные BCD, операнды или границы дают ошибку расширения. Команды добавлены для общей работы с таблицами; правила конкретной игры прошивка не интерпретирует.

2F: управление индикатором

Позиции `00`…`0B` — двенадцать знакомест слева направо: `00` — знак мантиссы, `01`…`08` — цифры мантиссы, `09` — знак порядка, `0A`…`0B` — цифры порядка.

Последовательность	Действие
`2F 00…0B`	Выбрать позицию
`2F 0D`	Очистить кадр и отключить автоматическое обновление, не меняя X
`2F 0E`	Вывести X; в сегментном виде записать маску и передвинуть курсор
`2F 25`	Передвинуть курсор без записи
`2F 50`	Отключить автоматическое обновление из X
`2F 52`	Вернуть автоматическое обновление
`2F 53`	Одновременно опубликовать все 12 масок из R0…R3
`2F 54`	Заменить одну маску в R0…R3 по упакованной точке X
`2F 55`	Стереть ту же клетку по упакованной точке X
`2F 56`	Собрать буфер R0…R3 по списку индексируемых слоёв из памяти программы
`2F 6n`	Записать восемь десятичных цифр Rn в буфер R0…R3
`2F 7n`	Записать шесть hex-цифр Rn в буфер R0…R3 по алфавиту из памяти программы
`2F 8n`	Загрузить 12 масок из памяти программы по адресу `RF + 12*Rn` в R0…R3
`2F 2A`	Переключиться на сегментный вид
`2F 30`	Вернуться к числовому виду

В сегментном виде целое X от 0 до 255 — маска: биты 0…6 задают сегменты A…G, бит 7 — точку. Все 12 позиций, включая знаковые 0 и 9, принимают произвольную маску. Автоматическая публикация X меняет текущую позицию без продвижения курсора; явный `2F 0E` продвигает его по кругу. X остаётся арифметическим регистром, независимо от состояния экрана.

`2F 0E` читает окончательный результат предыдущей команды в обоих видах индикатора; дополнительный NOP после арифметики не нужен. Это исправлено в исходниках 23.09.2026 (`091d96da`). Прежний обработчик мог, например, вывести маску 81 сразу после √81 вместо 9, ошибочно принять отрицательный результат за положительный или отклонить результат K [x] как дробный. Исправление использует уже существующий такт префикса и не добавляет команд в программу.

Пакетный кадр и быстрое форматирование

R0…R3 могут служить буфером кадра: каждый регистр содержит три маски в виде точного целого `a + 256*b + 65536*c` от 0 до 16777215. R0 соответствует позициям 0…2, R1 — 3…5, R2 — 6…8, R3 — 9…11. `2F 53` проверяет все четыре слова и заменяет весь сегментный кадр за одну команду. Повторная публикация одинакового кадра не вызывает перерисовку. Команда не меняет регистры, курсор, режим представления и автоматический вывод. Для удерживаемого сегментного экрана сначала задают `2F 50`, `2F 2A`.

2F 6n берёт целое Rn от 0 до 99999999 и записывает в буфер восемь десятичных цифр с ведущими нулями. 2F 7n берёт целое Rn от 0 до 16777215 и записывает шесть цифр по основанию 16. Их вид задаёт сама программа: RF содержит абсолютный адрес 16 последовательных байтов-масок для значений 0…15. Адрес таблицы — 0…9984; таблица может пересекать банк. Так hex-цифры могут отображаться произвольным алфавитом, включая точки.

Обе команды начинают с текущего курсора, продвигают его на ширину поля по кругу и сохраняют остальные позиции буфера. Они меняют только R0…R3 и курсор; X и стек сохраняются. Экран обновится лишь после 2F 53. Исходный Rn можно выбирать из всех шестнадцати регистров: если это R0…R3, его значение считывается до изменения буфера. При неверном числе, слове буфера или адресе таблицы команда завершается ошибкой расширения без частичной записи. Отрицательные и дробные значения не принимаются.

Например, при заполненном буфере 2F 02 2F 6C 2F 53 вставляет восемь цифр RC в позиции 2…9, сохраняя две позиции слева и две справа, затем публикует готовый кадр. Циклы деления на 10/256 в программе не нужны.

Замена отдельной клетки буфера

Для 2F 54/55 X — целое 256*позиция + маска, позиция 0…11, маска 0…255. Например, X=767 задаёт FF в позиции 2; 2F 54 заменяет её маску, 2F 55 записывает в эту клетку ноль. X=65535 — прозрачная точка: обе команды ничего не меняют. Её удобно возвращать из таблицы для объекта вне видимого участка. Другие значения выше 3071 недопустимы.

X, стек, RF, курсор и опубликованный экран сохраняются. Обычная точка проверяется вместе со всеми четырьмя словами буфера до записи; прозрачная точка не требует чтения буфера. Изменённый кадр выводится через 2F 53. Чтение таблиц и операции с клеткой учитывают завершение предыдущей команды ПЗУ, включая арифметику и запись регистра.

Пакетная сборка слоёв

2F 56 читает список по адресу RF и целиком заменяет R0…R3, сохраняя X, стек, остальные регистры, курсор и опубликованный экран. Первый байт — число слоёв 1…16, далее идут семибайтовые описатели:

```
режим, регистр-индекс, регистр-строка, делитель, шаг-строки, база-low, база-high
```

Режим 0 загружает запись из 12 масок, 1 заменяет клетку упакованной точкой, 2 стирает клетку. Первый слой обязательно имеет режим 0; последующие слои применяются по порядку. Точки имеют тот же формат, что у 2F 54/55, включая прозрачное значение 65535, и хранятся как 24-битные little-endian слова.

Регистр задаётся двоичным номером 0…15, его значение — неотрицательное целое меньше 10000. Номер исходной записи:

```
индекс + шаг-строки * floor(строка / делитель)
```

Для одномерной таблицы регистр-строка равен FF, делитель и шаг равны нулю. Для двумерной — регистр-строка 0…15, делитель 1…255, шаг 0…255. База — абсолютный двоичный little-endian адрес 0…9999. Размер записи — 12 байтов для режима 0 и 3 байта для режимов 1/2. Список и источники могут пересекать банки, но целиком должны помещаться в 0000…9999.

Например, слой {1,5,4,12,37,baseLo,baseHi} берёт точку из записи R5 + 37*floor(R4/12). Это обычная адресация массива, без правил конкретной игры. Все регистры читаются до записи результата: R0…R3 и RF допустимы как индексы. Ошибка даже в последнем слое сохраняет весь старый буфер. Отрисовка на экране выполняется отдельным 2F 53.

Загрузка готового сегментного кадра

2F 8n читает 12 последовательных байтов памяти программы, начиная с абсолютного адреса RF + 12*Rn. Каждый байт — маска одного знакоместа. Команда упаковывает их в R0…R3; предыдущие значения буфера не проверяются и полностью заменяются. Для вывода готового кадра затем выполняют 2F 53.

RF и Rn должны содержать неотрицательные целые меньше 10000, а последний байт записи должен находиться по адресу не выше 9999. Запись может пересекать границу банка; незаполненная память читается как нули. Все 16 вариантов Rn, включая R0…R3 и RF, допустимы: оба адресных операнда считываются до изменения буфера. Неверное число или выход за память вызывает ошибку расширения без

частичной записи.

X, стек, курсор, вид индикатора и автоматическое обновление сохраняются. Команда сама не публикует кадр. Например, при RF=896, R4=2 последовательность `2F 84 2F 53` выводит запись по адресам 920…931. Команда добавлена 29.10.2026; для таких программ нужна новая прошивка.

Произвольные сегменты рисуются на UC1609 с фиксированным калькуляторным шрифтом. На квалифицированном WS0010 они рисуются непосредственно в видимой графической области 80×16: все 256 масок различимы и ограничения CGRAM нет. Если графический режим WS0010 недоступен, используется тот же символьный алгоритм, что и на LCD1602 A00/A02. Двенадцать масок выводятся в двенадцать знакомест: знакомые цифры, буквы и знаки берутся из CGROM, остальные маски рисуются в 5×8 через CGRAM. В символьном сегментном режиме верхняя строка временно показывает `SEG`, а обычные мнемоники и обозначение угловой меры возвращаются после `2F 30`.

Маски без точки	Символы CGROM
`3F 06 5B 4F 66 6D 7D 07 7F 6F`	`0 1 2 3 4 5 6 7 8 9`
`77 7C 39 58 5E 79 71 76 74 1E 38`	`A b C c d E F H h J L`
`54 5C 73 50 3E 1C 6E`	`n o P r U u y`
`00 08 40 48`	пробел, `_`, `-`, `=`
`31 37`	только A02: Г П
`78`	A00: ROM-код `203` (`t`); A02: ASCII `t`
`C9`	только A00: ROM-код `208`, три горизонтали с точкой

Маска `80` выводится штатной точкой `.`. Остальные маски, включая цифры с точкой, получают точный пользовательский глиф; одна и та же маска во всех позициях занимает один слот. CGRAM вмещает только восемь разных глифов одновременно. Если кадр требует больше, наиболее часто повторяющиеся восемь остаются точными, остальные позиции показывают `?`, а верхняя строка — `SEG!`. Это ограничение символьного вывода, не ошибка исполнения программы: на UC1609 и графическом WS0010 тот же кадр остаётся точным.

Существуют и неиспользованные синонимы: A00-код `147` (оборотная E) и A02-буква Э похожи на маску `4F`, но она уже обозначает цифру `3`; A02-буква Ч похожа на маску `66`, но она уже обозначает цифру `4`. Поскольку сама маска не несёт признака «буква или цифра», в этих случаях цифры имеют приоритет. Код `95` (_) доступен в обеих CGROM.

Ввод из терминала

`hin` и `asm` принимают четырёхзначный абсолютный адрес в режимах `auto` и 112 шагов + RF; hout <адрес> <число-байтов> печатает произвольный диапазон как команды `hin`. В `asm` отдельный байт можно написать двумя hex-цифрами, даже если у него нет мнемоники. Одна строка записывает не более 112 байтов.

```
hin 0000 1F510250
hin 0250 0750
hout 0248 8
```

Команды очистки программы и `reinit` очищают и добавленные банки, освобождая все 32 слота для следующей программы. После очистки нулевые участки можно пропускать: `hin` меняет только указанные байты. Например, после `reinit` команда `hin 0250 0301` записывает два байта; `hout 0248 6` возвращает `hin 0248 000003010000`. Повторять `reinit` между частями загрузки нельзя: он удалит уже записанные части. Старые прошивки заполняли добавленные банки кодом `50`; для загрузчиков с пропущенными нулями нужна прошивка с новым нулевым заполнением (добавлено в сборке `89A59433`). Классический пошаговый редактор и `ins` не пересчитывают адреса внутри многобайтового `1F`; для таких программ используйте `hin`/`asm` и проверяйте байты через `hout`.